

CodeGraph Incorporation Plan

0. Scope & non-goals

This document is the **research deliverable for task CG1**. It plans the incorporation of the third-party tool **CodeGraph** (<https://github.com/colbymchenry/codegraph>) into the HelixCode project so that it is installed and verifiably working for **five CLI coding agents**:

1. **Claude Code** (primary)
2. **OpenCode**
3. **Kimi CLI**
4. **Crush**
5. **Qwen Code**

No software is installed and no commit is made by CG1. CG1 produces only this plan. The implementation phases (CG2...) follow this document.

1. What CodeGraph is

CodeGraph is a **pre-indexed code knowledge-graph engine** for AI coding agents. It parses a codebase into a local graph of symbols (functions, classes, methods) and edges (calls, imports, inheritance), stores that graph in a local SQLite database, and serves it to AI agents over the **Model Context Protocol (MCP)** — so an agent queries the graph instantly instead of spawning grep/glob/Read scans. The project's headline claim is "92% fewer tool calls · 71% faster · 100% local".

Sources (fetched 2026-05-20):

- README: <https://github.com/colbymchenry/codegraph> (repo description: *"Pre-indexed code knowledge graph for Claude Code, Codex, Cursor, and OpenCode — fewer tokens, fewer tool calls, 100% local"*).
- npm package: <https://www.npmjs.com/package/@colbymchenry/codegraph> (version 0.7.11 at research time).
- Repo file tree via `gh api repos/colbymchenry/codegraph/contents — package.json,src/{installer,mcp,bin,extraction,...},docs/,scripts/local-install.sh`.

1.1 Technology & runtime

- **Language**: TypeScript (~94%), compiled to dist/ JavaScript.
- **Runtime**: Node.js >=18 <25 (package.json engines). HelixCode host has Node v22.19.0 / npm 10.9.3 — compatible.

- **Parsing:** web-tree-sitter + tree-sitter-wasms (WASM tree-sitter; no native compile needed). Native better-sqlite3 is an *optional* dependency; node-sqlite3-wasm is the always-present fallback — so install works without a C toolchain.
- **Storage:** SQLite DB at <project>/codegraph/codegraph.db, FTS5 index.
- **License:** MIT.

1.2 Languages & frameworks indexed

19+ languages including **Go** (HelixCode's primary language), TypeScript, JavaScript, Python, Rust, Java, C#, PHP, Ruby, C, C++, Swift, Kotlin, Dart. Framework routing recognition for 13 frameworks including **Gin** (the HelixCode HTTP framework) — so CodeGraph can map HelixCode's Gin routes to their handlers.

1.3 Install / init / scan / query model

- **Install:** npx @colbymchenry/codegraph (interactive installer) or npm install -g @colbymchenry/codegraph (global codegraph binary). The installer auto-detects agents and writes their MCP config.
- **Initialize a project:** cd <project> && codegraph init -i — creates .codegraph/ (DB + config.json); -i runs the first index immediately.
- **Scan:** tree-sitter extraction → SQLite storage → reference resolution → filesystem-watch auto-sync (2 s debounce).
- **Query (CLI):** codegraph query <search>, codegraph context <task>, codegraph affected [files], codegraph status [path], codegraph files, codegraph index, codegraph sync, codegraph serve --mcp.
- **Query (MCP tools exposed to agents):** codegraph_search, codegraph_context, codegraph_callers, codegraph_callees, codegraph_impact, codegraph_node, codegraph_files, codegraph_status.

1.4 Per-agent MCP server config shape (from src/installer/)

The shared stdio server entry (src/installer/targets/shared.ts getMcpServerConfig()):

```
{ "type": "stdio", "command": "codegraph", "args": ["serve", "--mcp"] }
```

OpenCode variant (src/installer/targets/opencode.ts) collapses binary+args into one array: { "type": "local", "command": ["codegraph","serve","--mcp"], "enabled": true } under an mcp.<name> wrapper.

2. Recommended incorporation approach

Recommendation: vendored npm install pinned + thin wrapper, NOT a git submodule.

Rationale (constitution-aligned):

- **§11.4.74 submodule-catalogue-first / CONST-050 no-reimplementation:** CodeGraph is a third-party MIT tool. It is *incorporated as an external dependency*, never reimplemented. ✓
- **Not a submodule:** CodeGraph publishes to npm and ships pre-built dist/. Adding the GitHub repo as a git submodule would require building TypeScript in-tree and tracking upstream churn for zero benefit — HelixCode does not modify CodeGraph source. CONST-051(B) also forbids injecting HelixCode context into a dependency; a submodule invites exactly that. A pinned npm install keeps CodeGraph fully decoupled and project-not-aware.
- **CONST-038 project-tightening / Git-remote policy:** a submodule must be a vasic-digital/* or HelixDevelopment/* remote; colbymchenry/codegraph is neither. A submodule is therefore *not permitted* anyway. npm install sidesteps the constraint cleanly.
- **CONST-045-style host independence:** pin the version in a manifest so the install is reproducible and not a moving target.

2.1 Location in the HelixCode tree

Create a new top-level tooling directory (lowercase snake_case per CONST-052):

```
HelixCode/
├─ tools/
│   └─ codegraph/
│       ├── README.md           # what it is, how HelixCode
uses it
│       ├── codegraph.version   # pinned version string, e.g.
0.7.11
│       ├── install.sh          # idempotent installer (see §3)
│       └─ verify.sh            # anti-bluff end-to-end proof
(see §5)
└─ agents/                     # per-agent registration
helpers
    ├── register-claude.sh
    ├── register-opencode.sh
    ├── register-kimi.sh
    ├── register-crush.sh
    └─ register-qwen.sh
```

- tools/ is new; it does not collide with the root internal/ / cmd/ security_test/ helpers described in CLAUDE.md §3.2.
- The CodeGraph runtime itself is installed into a project-local tools/codegraph/node_modules/ (via npm install --prefix) so nothing leaks into the host's global npm and the install is reproducible.
- The scanned-graph artefact .codegraph/ is created at the **HelixCode repo root** (and inside helix_code/ for the inner Go module — see §3.4).

2.2 .gitignore obligations (CONST-053)

The following **MUST** be added to the relevant .gitignore files **before** any implementation commit:

- tools/codegraph/node_modules/ — build/dependency artefact, forbidden.
 - .codegraph/ — generated SQLite DB + index, fully recreatable by codegraph init -i, forbidden from version control.
 - tools/codegraph/codegraph.version and the .sh scripts **ARE** committed (they are the generator/source per CONST-053 recreatable-content test).
-

3. Install + initialize + scan procedure

All steps are codified into tools/codegraph/install.sh (idempotent) during implementation. Procedure:

3.1 Preconditions

- Node.js >=18 <25 on PATH (host has v22.19.0 ✓).
- npm on PATH (host has 10.9.3 ✓).

3.2 Install CodeGraph (pinned)

```
cd HelixCode/tools/codegraph
npm install --prefix . @colbymchenry/codegraph@$(cat
    codegraph.version)
# exposes ./node_modules/.bin/codegraph
```

Do **not** use the upstream interactive npx @colbymchenry/codegraph installer for the agent wiring — it only knows Claude/Cursor/Codex/OpenCode and would write configs HelixCode wants to control explicitly. Use it (if at all) only for the binary install; do the five-agent wiring with HelixCode's own agents/register-*.sh scripts (§4).

3.3 Make the binary discoverable

The MCP entries reference the bare command codegraph. Two options — pick one in implementation:

- **(a)** symlink tools/codegraph/node_modules/.bin/codegraph into a directory already on the operator's PATH; or
- **(b)** use an **absolute path** to node_modules/.bin/codegraph as the command in every agent's MCP config (more reproducible, no PATH mutation — **preferred**).

3.4 Initialize + scan HelixCode

HelixCode is a meta-repo with an inner Go module. Initialize **both**:

config.json languages should include go (auto-detected). exclude should list node_modules/**, bin/**, dist/**, dependencies/**, cli_agents_resources/** to keep the index focused on first-party code.

```
codegraph status .           # expect non-zero node/edge/file
    counts                  # expect real HelixCode symbols
codegraph query Provider    # expect real HelixCode symbols
    (e.g. the               # llm.Provider interface) – non-
    empty
```

4. Per-agent integration sub-plans

4.1 Claude Code (primary)

- **Config file:** project-local `HelixCode/.mcp.json` (preferred for a shared, version-controllable project config) — CodeGraph's own installer uses `~/ .claude.json / ./ .claude.json`, but the modern Claude Code project convention is `.mcp.json`.
- **Registration (CLI, preferred):**

- **Equivalent JSON (.mcp.json):**

- **Permissions:** add `mcp__codegraph_*` to `.claude/settings.json` permissions.allow so the eight tools run without prompts.
- **Source:** `src/installer/targets/claude.ts` (~/.claude.json [?](#))
`./claude.json`, `settings.json` permission shape
`mcp__codegraph__<tool>`).

4.2 OpenCode

- **Config file:** project-local HelixCode/opencode.jsonc (OpenCode reads .jsonc; global is ~/.config/opencode/opencode.jsonc).
- **Shape** (note: mcp.<name>, command is an array, enabled flag):

```
{ "$schema": "https://opencode.ai/config.json",  
  "mcp": { "codegraph": { "type": "local",  
    "command": ["<CG>", "serve", "--mcp"], "enabled": true } } }
```

- Source: src/installer/targets/opencode.ts, <https://opencode.ai/docs/config>.

4.3 Kimi CLI

- **Config file:** ~/.kimi/mcp.json (Claude-Desktop-compatible mcpServers shape). Project-scoped alternative: a dedicated file passed with `kimi --mcp-config-file <path>`.
- **Registration (CLI):** `kimi mcp add` (interactive), or write directly:

```
{ "mcpServers": { "codegraph": {  
  "command": "<CG>", "args": ["serve", "--mcp"] } } }
```

- Sources: <https://moonshotai.github.io/kimi-cli/en/customization/mcp.html>, <https://deepwiki.com/MoonshotAI/kimi-cli/6.7-mcp-integration>, <https://github.com/MoonshotAI/kimi-cli>.

4.4 Crush

- **Config file:** project-local HelixCode/.crush.json (load priority .crush.json → crush.json → ~/.config/crush/crush.json).
- **Shape** (Crush uses an mcp wrapper, type: "stdio"):

```
{ "$schema": "https://charm.land/crush.json",  
  "mcp": { "codegraph": { "type": "stdio",  
    "command": "<CG>", "args": ["serve", "--mcp"] } } }
```

- Security note: Crush treats crush.json as trusted code (`$(...)` runs at load time) — use a literal absolute path, no shell expansion.
- Sources: <https://github.com/charmbracelet/crush>, <https://brightdata.com/blog/ai/crush-cli-with-web-mcp>.

4.5 Qwen Code

- **Config file:** project scope HelixCode/.qwen/settings.json (user scope ~/.qwen/settings.json).
- **Shape** (top-level mcpServers, stdio):

```
{ "mcpServers": { "codegraph": {  
  "command": "<CG>", "args": ["serve", "--mcp"], "timeout":  
    15000 } } }
```

- **Registration (CLI):** `qwen mcp manage` command.

- Sources: <https://qwenlm.github.io/qwen-code-docs/en/developers/tools/mcp-server/>, <https://github.com/QwenLM/qwen-code/blob/main/docs/users/features/mcp.md>.

4.6 Cross-agent summary table

Agent	Config file (project scope)	Wrapper key	command form
Claude Code	.mcp.json	mcpServers	command + args
OpenCode	opencode.jsonc	mcp	command[] array
Kimi CLI	~/.kimi/mcp.json	mcpServers	command + args
Crush	.crush.json	mcp	command + args
Qwen Code	.qwen/settings.json	mcpServers	command + args

All five share the same stdio server process: `<CG> serve --mcp`.

5. Anti-bluff test plan (CONST-035 — every PASS carries runtime evidence)

"Installed" is **not** "working". Per CONST-035 / Article XI §11.9, every PASS below **MUST** capture and paste real runtime output. Three layers:

5.1 Layer A — CodeGraph itself works on the real HelixCode repo

Challenge CG-CHALLENGE-01 (challenges/ bank or helix_code/tests/e2e/challenges/):

1. Run `codegraph status .` at the HelixCode root. **PASS only if** the JSON reports `initialized:true` AND `files > 0` AND `nodes > 0` AND `edges > 0`. Capture the JSON.
 2. Run `codegraph query Provider`. **PASS only if** the result set is non-empty AND contains a real HelixCode symbol path (e.g. a hit under `helix_code/internal/llm` or `internal/provider`). Capture the output.
 3. Run `codegraph context "add a new LLM provider"`. **PASS only if** the built context references real HelixCode files. Capture the output.
- **Anti-bluff guard:** assert the output does NOT contain simulated, placeholder, TODO, or an empty/zero result. A zero-node graph is a FAIL, not a PASS.

5.2 Layer B — the MCP server responds (transport-level)

Challenge CG-CHALLENGE-02:

- Launch `<CG> serve --mcp`, send a JSON-RPC `initialize` then a `tools/list` request over stdio, assert the response lists all eight `codegraph_*` tools.

- Send a `tools/call` for `codegraph_search` with `{"query": "Provider"}` and assert a non-empty real result. Capture the JSON-RPC wire exchange — this is the captured-wire-evidence CONST-050(B) requires.

5.3 Layer C — each of the five agents actually reaches CodeGraph

One Challenge per agent (CG-CHALLENGE-03..07). For each agent:

1. Register CodeGraph via §4's procedure into a throwaway copy of the agent config.
2. Start the agent and run its "list MCP tools / list MCP servers" command (`claude mcp list`, `qwen mcp`, `kimi mcp list`, OpenCode/Crush startup MCP log). **PASS only if** `codegraph` appears AND its tools are listed.
3. Drive the agent (non-interactively where the agent supports it) with a prompt that can only be answered from the graph — e.g. *"Using the codegraph tools, list the callers of "* — and assert the agent's answer contains the real caller(s). Capture the transcript.

- **Anti-bluff guard:** a config file that merely *contains* the `codegraph` entry is NOT a PASS. The PASS bar is the agent invoking a `codegraph_*` tool and returning real graph data. Config-only / absence-of-error PASS is a CONST-035 critical defect.

5.4 helix_qa integration (CONST-050(B))

Register CG-CHALLENGE-01..07 as a test bank in `helix_qa` so an autonomous QA session executes them with per-check wire evidence. Coverage-ledger entry (CONST-048): feature = "CodeGraph integration", platform-axis = the five agents, invariants 1–6.

6. Phased task breakdown (subagent-sized)

CG1 (this document) is **done** on write. Implementation follows:

Phase A — incorporation scaffolding

- **CG2:** Create `tools/codegraph/` skeleton (`README.md`, `codegraph.version` pinned to 0.7.11, empty `install.sh/verify.sh`, `agents/` dir). Add `.gitignore` rules for `tools/codegraph/node_modules/` and `.codegraph/` (CONST-053).
- **CG3:** Write `tools/codegraph/install.sh` — idempotent pinned npm `install --prefix`, Node-version precheck, absolute-path resolution for the binary (§3.2–3.3). Run it; capture output.

- **CG4:** Initialize + scan — run `codegraph init -i` at repo root and in `helix_code/`; tune `.codegraph/config.json` exclude list (§3.4). Capture `codegraph status` JSON as evidence.

Phase B — per-agent registration

- **CG5:** `agents/register-claude.sh` + write `.mcp.json` entry + `mcp__codegraph__*` permission. Capture `claude mcp list`.
- **CG6:** `agents/register-opencode.sh` + `opencode.jsonc` entry.
- **CG7:** `agents/register-kimi.sh` + `~/kimi/mcp.json` entry.
- **CG8:** `agents/register-crush.sh` + `.crush.json` entry.
- **CG9:** `agents/register-qwen.sh` + `.qwen/settings.json` entry.

Phase C — anti-bluff verification

- **CG10:** Write `tools/codegraph/verify.sh` + Challenge CG-CHALLENGE-01 (Layer A — CodeGraph works on real repo). Capture evidence.
- **CG11:** Challenge CG-CHALLENGE-02 (Layer B — MCP transport / JSON-RPC wire test). Capture evidence.
- **CG12:** Challenges CG-CHALLENGE-03..07 (Layer C — one per agent, end-to-end agent → codegraph tool call). Capture five transcripts.
- **CG13:** Register the seven Challenges as a `helix_qa` test bank; run an autonomous QA session; capture per-check wire evidence (CONST-050(B)).

Phase D — governance & documentation

- **CG14:** QWEN.md anti-bluff covenant gap fix (see §7).
- **CG15:** Document CodeGraph in HelixCode docs — `tools/codegraph/README.md`
 - a reference in `docs/COMPLETE_CLI_REFERENCE.md`; update `docs/CONTINUATION.md` (CONST-044) and the coverage ledger (CONST-048).
- **CG16:** CONST-047 recursive sweep — assess whether any owned submodule (`helix_qa`, `challenges`, `containers`, ...) also benefits from a CodeGraph config and, if so, apply the same wiring; otherwise record the assessment.

Implementation task count: 15 tasks (CG2...CG16).

7. Anti-bluff covenant cascade — QWEN.md gap assessment

The operator requires the anti-bluff covenant to be present in the project's Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md, and all submodules.

Assessment from the current tree:

- CONSTITUTION.md, CLAUDE.md, AGENTS.md at the HelixCode root all carry the anti-bluff covenant (CONST-035, Article XI §11.9, CONST-050) and the full CONST-0xx cascade — **present**.
- CRUSH.md is present at the root (per CLAUDE.md §1.1 peer-governance list) — needs a content check during CG14 to confirm the covenant text is current.
- QWEN.md **exists** at the root (/run/media/.../HelixCode/QWEN.md, 24 971 bytes, last modified 2026-05-16) but is **older** than CLAUDE.md / AGENTS.md (both 2026-05-20). It is therefore the **likely cascade gap** — it may pre-date the most recent CONST-0xx anchors. CG14 MUST diff QWEN.md (and CRUSH.md) against the current CLAUDE.md/AGENTS.md anti-bluff and CONST-0xx sections and bring them into sync.
- The constitution submodule (constitution/) §11.4 covenant is already cascaded into submodules per the existing close-out history; CG16's recursive sweep MUST still re-verify with scripts/verify-governance-cascade.sh after the CodeGraph work lands.

Conclusion: the covenant is broadly cascaded; **QWEN.md is the most probable gap** and CRUSH.md needs a confirmation diff. Both are handled in CG14, gated by the cascade verifier in CG16.

8. Risks & open questions

1. **PATH vs absolute path** — §3.3: prefer absolute paths in MCP config to avoid mutating the operator's shell profile (CONST-053-adjacent hygiene).
 2. **Cursor's --path workaround** — src/installer/targets/cursor.ts shows Cursor needs an explicit --path arg because it does not pass workspaceFolders. None of HelixCode's five target agents is Cursor, but if any of the five also omits the workspace root in the MCP initialize call, CodeGraph's process.cwd() fallback may miss .codegraph/. CG5–CG9 MUST verify each agent's working directory and add --path <repo-root> to args if a "not initialized" error appears.
 3. **Inner-module duality** — HelixCode has two graphs (root + helix_code/). Decide in CG4 whether agents point at the root graph, the inner graph, or both (multiple registered MCP servers). Recommendation: index the inner Go module (the real domain code) as the primary; index root for governance tooling if useful.
 4. **Upstream version drift** — pin in codegraph.version; bumping it is an explicit, reviewed change (re-run all CG-CHALLENGE-* on bump).
-

9. Sources (all fetched 2026-05-20)

- CodeGraph repo: <https://github.com/colbymchenry/codegraph>
- CodeGraph npm: <https://www.npmjs.com/package/@colbymchenry/codegraph>
- CodeGraph source inspected via `gh api repos/colbymchenry/codegraph/contents — package.json, src/installer/targets/{claude,opencode,codex,cursor,shared}.ts, src/bin/codegraph.ts, scripts/local-install.sh`.
- Kimi CLI MCP: <https://moonshotai.github.io/kimi-cli/en/customization/mcp.html>, <https://deepwiki.com/MoonshotAI/kimi-cli/6.7-mcp-integration>, <https://github.com/MoonshotAI/kimi-cli>
- Qwen Code MCP: <https://qwenlm.github.io/qwen-code-docs/en/developers/tools/mcp-server/>, <https://github.com/QwenLM/qwen-code/blob/main/docs/users/features/mcp.md>
- Crush MCP: <https://github.com/charmbracelet/crush>, <https://brightdata.com/blog/ai/crush-cli-with-web-mcp>
- OpenCode config: <https://opencode.ai/docs/config>

! §11.4.99 STALENESS FLAG — CodeGraph upstream has materially changed since this plan was written (verify-recheck 2026-06-16)

This CG1 research deliverable was written **2026-05-20 against CodeGraph 0.7.11**. A §11.4.99 latest-source re-verification on **2026-06-16** found the upstream tool has **materially changed**. Treat the §1–§4 install/version specifics below as **STALE** until the implementation phases (CG2...) re-pin against the values confirmed here. The §4 *per-agent MCP config shapes* (OpenCode/Qwen) remain accurate (re-confirmed below); the **CodeGraph package itself** is what drifted:

Fact	This plan (2026-05-20, v0.7.11)	Current official (2026-06-16)	Action for CG2/CG3
Latest npm version	0.7.11	1.0.1 (released 2026-06-13)	Re-pin codegraph.version to 1.0.1 (re-run all CG- CHALLENGE-* on bump per §8.4)
Node engines	>=18 <25	>=20 <25, and the package now bundles its own runtime (self- contained)	Host has Node v22.19.0 ✓ still in range; bundled- runtime means the §3.1 Node precheck is now belt- and-suspenders, not load- bearing

Fact	This plan (2026-05-20, v0.7.11)	Current official (2026-06-16)	Action for CG2/CG3
Headline claim	"92% fewer tool calls · 71% faster · 100% local"	"~16% cheaper · ~58% fewer tool calls · 100% local"	§1's quoted headline is outdated marketing copy — do not cite the old numbers to the operator
Install model	npm install --prefix / interactive npx	Official one-liner installers now exist: curl -fsSL .../ install.sh sh (macOS/Linux) + install.ps1 (Windows); npm i -g @colbymchenry/ codegraph still works	§3.2's pinned npm install --prefix approach is still valid and remains preferred for reproducibility, but it is no longer the <i>only</i> sanctioned path
Agent auto-wiring	upstream installer knows Claude/Cursor/Codex/OpenCode only	codegraph install now auto-detects Claude Code, Cursor, Codex, opencode, Hermes Agent, Gemini CLI, Antigravity, Kiro	Kimi CLI, Crush, and Qwen Code (3 of this plan's 5 targets) are STILL NOT in upstream auto-detect → this plan's §4 manual per-agent wiring for those three remains REQUIRED, not superseded

Why this matters (§11.4.99): an operator following §1–§3 verbatim would npm install the wrong pinned version (0.7.11) and quote stale performance numbers. None of the drift is a *safety/ban* hazard (unlike the Telegram-VoIP case), but it is a correctness staleness that CG2/CG3 MUST resolve before this plan is treated as authoritative. The §4 config-shape instructions were re-confirmed correct and are NOT stale.

Sources verified 2026-06-16:

CodeGraph package + install/CLI/agent-wiring facts (re-verified — see staleness flag above):

- CodeGraph npm registry (latest 1.0.1, engines node >=20 <25, bundled runtime, bin: codegraph): <https://registry.npmjs.org/@colbymchenry/codegraph>
- CodeGraph repo README (one-liner installers, codegraph install agent auto-detect list, init/index/sync/serve --mcp CLI, "~16% cheaper · ~58% fewer tool calls" headline, v1.0.1 2026-06-13): <https://github.com/colbymchenry/codegraph>

Per-agent MCP config shapes (re-confirmed CURRENT — no change needed):

- OpenCode local MCP server shape (mcp.<name> wrapper, type:"local", command array, enabled flag — matches §4.2): <https://opencode.ai/docs/mcp-servers>, <https://opencode.ai/docs/config>
- Qwen Code MCP shape (top-level mcpServers, command+args+timeout, qwen mcp add/manage CLI — matches §4.5): <https://qwenlm.github.io/qwen-code-docs/en/developers/tools/mcp-server/>

Not re-fetched this round (cited at original-research time 2026-05-20, §9 above; re-verify before CG7/CG8 implementation): Kimi CLI MCP docs, Crush MCP docs.